

A UNIFIED REVIEW AND ENGINEERING FRAMEWORK

Intelligent Object Engineering

*Integrating Embedded Intelligence, Connected Systems,
Mathematical Modelling, and Edge Computer Vision*

From sensing and computation to inference, decision, communication, and action

Academic review manuscript — Digital Engineering perspective

Komsan Kanjanasit, PhD

College of Computing, Prince of Songkla University, Phuket Campus

September 2026

Abstract

Intelligent Object Engineering (IOE) is proposed in this manuscript as an interdisciplinary engineering framework for designing physical objects that can sense, compute, communicate, infer, decide, and act within real environments. It extends conventional embedded-system and Internet of Things development by treating intelligence, physical interaction, networking, trustworthiness, and lifecycle assurance as co-engineered properties rather than separate add-on functions. The framework positions an intelligent object as a cyber-physical unit in which sensors acquire observations, embedded computation transforms data into state estimates, artificial intelligence or analytical models infer context, decision logic selects an action, actuators influence the environment, and communication enables coordination with other objects, edge nodes, digital twins, and cloud services. The manuscript develops a six-layer reference architecture, describes a capability maturity model, and links IOE to its mathematical foundations in calculus, linear algebra, probability, signal processing, state estimation, optimization, control, information theory, graph theory, and machine learning. Particular attention is given to image processing and embedded computer vision because a camera functions as a high-dimensional sensor that enables visual perception. An ESP32-class intelligent plant-monitoring object is used to illustrate requirements, models, data flow, inference, actuation, connectivity, security, and validation. The resulting perspective provides a coherent basis for research, system design, and computer-engineering education in intelligent connected and embedded systems.

Keywords—intelligent objects; embedded intelligence; connected intelligence; Internet of Things; cyber-physical systems; edge AI; TinyML; image processing; embedded vision; sensor fusion; control systems; digital twins.

1. Introduction

The evolution of embedded computing has transformed physical products from isolated electronic devices into connected, context-aware, and increasingly autonomous systems. Sensors provide observations of the physical world; microcontrollers and edge processors execute real-time algorithms; communication interfaces connect objects to services and peers; and machine-learning models enable classification, prediction, anomaly detection, and adaptive behavior. Yet these capabilities are often studied independently under embedded systems, IoT, cyber-physical systems (CPS), edge computing, artificial intelligence, robotics, or computer vision. A unified engineering view is therefore useful when the primary design unit is the physical object itself.

This manuscript uses the term Intelligent Object Engineering (IOE) as an umbrella framework for the systematic design, implementation, integration, validation, deployment, and lifecycle management of physical objects with embedded and connected intelligence. The term is not presented as a replacement for established CPS or IoT terminology. Instead, it provides an object-centred engineering lens that combines their foundational concepts with edge intelligence, perception, decision, and actuation. This framing is consistent with smart-object research and with CPS/IoT frameworks that integrate logical, physical, transducing, and human components [1]–[4].

The central proposition is that an intelligent object is more than a networked sensor and more than an AI model placed on a microcontroller. It is a constrained cyber-physical system whose intelligence must remain dependable under limitations in energy, memory, processing capacity, communication bandwidth, time, uncertainty, and security. Accordingly, IOE asks not only whether an algorithm is accurate, but also whether the complete object responds on time, operates safely, protects data,

fails predictably, and continues to deliver its intended physical function.

1.1 Scope and contributions

The manuscript makes four conceptual contributions. First, it formulates an operational definition and capability model for intelligent objects. Second, it organizes IOE using a layered architecture and a closed-loop engineering model. Third, it unifies the principal mathematics that connect physical processes, sensor data, estimation, learning, communication, and control. Fourth, it establishes image processing and embedded vision as the visual-perception branch of IOE and relates perception outputs to decisions, actuation, and connected services.

2. Foundations of Intelligent Object Engineering

2.1 Operational definition

An intelligent object is a physical artifact equipped with sensing, embedded computation, decision capability, optional learning or adaptation, communication, and—where the application requires it—actuation. Its purpose is to maintain an operational relationship with its environment rather than merely store or transmit data.

Intelligent Object = Physical Object+Sensing+Computing+Connectivity+Intelligence+Actuation.

Not every implementation must use a neural network. Intelligence may arise from rules, finite-state logic, model-based estimation, optimization, probabilistic inference, classical machine learning, or deep learning. The engineering criterion is whether the object can transform observations and context into useful, situation-dependent decisions under its resource and assurance constraints.

2.2 From embedded object to collaborative intelligent system

Table 1: Capability maturity model for intelligent objects

Level	Capability	Representative behavior
0	Passive	Identified or observed but performs no sensing or reasoning; e.g., a passive RFID tag.
1	Sensing	Measures one or more physical variables and makes data locally available.
2	Connected	Exchanges measurements, state, or commands through a wired or wireless network.
3	Reactive	Applies deterministic rules or control logic to respond to events.
4	Intelligent	Uses inference, prediction, classification, or model-based reasoning.
5	Adaptive	Updates parameters, models, or policies as conditions change.
6	Collaborative	Coordinates decisions or actions with multiple intelligent objects and services.

The levels are cumulative in capability but not prescriptive in architecture. A safety-critical Level 3 controller may be more valuable and trustworthy than an unnecessarily adaptive Level 5 device. Maturity should therefore be assessed against mission requirements rather than equated with algorithmic complexity.

2.3 Embedded intelligence and connected intelligence

Embedded intelligence places inference or reasoning within the object or immediately adjacent edge hardware. Connected intelligence emerges when objects, gateways, services, digital twins, or human operators exchange data and coordinate decisions. These forms complement rather than replace each other.

$$\text{Embedded intelligence: } \mathbf{x}_k \rightarrow f(\mathbf{x}_k; \boldsymbol{\theta}) \rightarrow \hat{y}_k \rightarrow a_k.$$

$$\text{Connected intelligence: } O_1 \leftrightarrow O_2 \leftrightarrow \dots \leftrightarrow O_N.$$

Local execution can reduce latency, network dependence, and raw-data transmission, whereas connected execution can provide greater compute capacity, fleet-wide knowledge, coordinated behavior, and model updates. The allocation of functions across device, edge, and cloud is therefore a system-design decision.

3. Reference Architecture

3.1 Six engineering layers

Table 3: Layered IOE reference architecture

Layer	Primary role	Examples
Physical	Defines the object, environment, energy flow, and physical constraints.	Machine, room, body, vehicle, plant, battery.
Sensing and actuation	Transduces physical variables and applies physical actions.	Temperature sensor, IMU, camera, motor, relay, pump.
Embedded computing	Acquires data and executes deterministic real-time functions.	ESP32, STM32, RTOS, drivers, local storage.
Intelligence	Estimates state, recognizes patterns, predicts outcomes, and selects actions.	Kalman filter, rules, TinyML, optimization.
Connectivity	Exchanges observations, state, commands, and models.	BLE, Wi-Fi, MQTT, CoAP, gateways.
Application and lifecycle	Provides human interaction, fleet management, digital twins, updates, and evidence.	Dashboard, alerts, OTA, audit logs, digital twin.

3.2 Closed-loop operation

The defining operational structure is a closed loop. The environment influences sensors; the object converts sensor output into observations; computational models estimate the state; decision logic produces a command; and actuators change the environment. Communication can enter or leave the loop at several points.

Environment $\rightarrow$ Sense $\rightarrow$ Process $\rightarrow$ Infer $\rightarrow$ Decide $\rightarrow$ Act $\rightarrow$ Environment.

Let $\mathbf{x}_k$ denote the physical state, $\mathbf{u}_k$ the control input, $\mathbf{z}_k$ the sensor measurement, and $\mathbf{y}_k$ the observable output. A general discrete-time model is

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k), \quad \mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k), \quad \mathbf{u}_k = \pi(\mathbf{z}_{0:k}, \mathbf{c}_k).$$

where $\mathbf{w}_k$ and $\mathbf{v}_k$ describe process and measurement uncertainty, π is the decision policy, and $\mathbf{c}_k$ is contextual information such as time, user intent, network state, or mission constraints. This compact model connects physical dynamics, sensing, estimation, intelligence, and control.

3.3 Cross-cutting engineering properties

- Timing: sampling periods, deadlines, communication delay, synchronization, and control-loop stability.
- Resources: energy, compute cycles, memory, storage, bandwidth, and thermal limits.
- Trustworthiness: security, privacy, safety, reliability, resilience, explainability, and maintainability.
- Interoperability: data models, interfaces, discovery, semantics, and version compatibility.
- Lifecycle: provisioning, calibration, monitoring, model updates, over-the-air firmware, decommissioning, and evidence retention.

4. Mathematical Foundations

Mathematics provides the bridge between the physical object and its computational representation. IOE requires a compact but integrated foundation rather than isolated mathematical topics.

Table 5: Mathematical domains and their roles in IOE

Domain	Core concepts	IOE use
Calculus and differential equations	Rates, integrals, continuous dynamics	Motion, thermal behavior, motors, energy and digital twins.
Linear algebra	Vectors, matrices, transformations, eigenstructure	IMUs, coordinate frames, sensor fusion, vision and neural networks.
Probability and statistics	Random variables, noise, estimation, confidence	Sensor uncertainty, anomaly detection and probabilistic decisions.
Signals and systems	Sampling, filtering, convolution, spectra	Conditioning IMU, audio, vibration and image signals.

Domain	Core concepts	IOE use
State estimation	State-space models, Bayesian filters	Recovering hidden state from noisy measurements.
Control theory	Feedback, stability, PID and optimal control	Safe and timely actuation.
Optimization	Objectives, constraints, trade-offs	Energy, latency, accuracy, routing and model compression.
Information and graph theory	Entropy, mutual information, nodes and edges	Communication, feature selection and collaborative objects.
Machine learning	Loss functions, gradients, generalization	Prediction, classification and adaptive behavior.

4.1 Physical dynamics and discrete computation

Many intelligent objects interact with systems whose state changes continuously. A mass–spring–damper model illustrates the relationship between force and motion:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = F(t).$$

Embedded software executes at discrete instants. A simple numerical update is

$$x_{k+1} = x_k + \Delta t v_k, \quad v_{k+1} = v_k + \Delta t a_k.$$

Sampling rate and numerical method are engineering choices: an interval that is too large can introduce delay, instability, or modelling error, while an unnecessarily small interval increases energy and compute demand.

4.2 Linear algebra and sensor representation

Multiaxis sensors naturally produce vectors. An inertial measurement unit can be represented by

$$\mathbf{s}_k = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^\top.$$

Coordinate transformations, calibration, feature extraction, and neural-network layers use matrix operations. A typical layer is

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}).$$

4.3 Probability, uncertainty, and state estimation

A sensor does not directly reveal the true physical state. A common measurement model is $\mathbf{z}_k = \mathbf{x}_k + \mathbf{v}_k$, where $\mathbf{v}_k$ may be approximated as Gaussian noise. Mean, variance, confidence intervals, and likelihoods provide a language for uncertainty. A linear state-space model is

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k.$$

This model underlies the Kalman filter and many sensor-fusion techniques. For nonlinear systems, extended, unscented, or particle filters may be more appropriate. The estimator should report uncertainty whenever the decision consequence depends on confidence.

4.4 Signal processing

Raw sensing often requires filtering before inference. A moving-average filter is

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k].$$

Frequency-domain analysis uses the discrete Fourier transform

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}.$$

Filter choice must reflect signal bandwidth, sampling rate, acceptable delay, numerical precision, and MCU resources. In intelligent objects, filtering is part of the inference chain and should be validated with the deployed model rather than as an independent preprocessing step.

4.5 Control and optimization

When an object acts, decision quality must be connected to physical response. With error $e(t) = r(t) - y(t)$, a proportional–integral–derivative controller is

$$u(t) = K_P e(t) + K_I \int e(t) dt + K_D \frac{de(t)}{dt}.$$

Resource allocation and design trade-offs can be written as constrained optimization:

$$\min J = \alpha E + \beta D + \gamma L, \quad \text{s.t.} \quad M \leq M_{\max}, \quad D \leq D_{\max}, \quad \text{safety constraints.}$$

Here E is energy, D is delay, and L is inference loss. Weights encode priorities, but safety constraints should generally be enforced explicitly rather than traded away through a small penalty.

4.6 Machine learning and information

For supervised learning, a model f_{θ} is trained from examples $(\mathbf{x}_i, y_i)$ by minimizing an empirical loss:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(\mathbf{x}_i)).$$

Entropy $H(X) = -\sum_x p(x) \log_2 p(x)$ measures uncertainty, while mutual information $I(X;Y)$ measures statistical dependence. These quantities support sensor selection, feature evaluation, compression, and communication-efficient inference. Graph $G = (V, E)$ represents a connected system in which vertices are objects or services and edges are communication or interaction links.

5. Image Processing and Visual Intelligence

5.1 A camera as a high-dimensional sensor

Image processing is the visual-sensing branch of IOE. A scalar sensor may produce one measurement, whereas a camera produces a spatial field $I(x, y)$, or a color vector $[R(x, y), G(x, y), B(x, y)]^T$ at every pixel. This large observation space contains appearance, geometry, motion, and contextual

information, but it also creates substantial compute, memory, bandwidth, privacy, and illumination challenges.

Physical scene $\rightarrow$ Image sensor $\rightarrow$ Preprocessing $\rightarrow$ Vision inference $\rightarrow$ Decision $\rightarrow$ Action.

5.2 Image processing versus computer vision

Image processing transforms or improves image data; computer vision extracts semantic or geometric meaning. The boundary is not absolute, but it is operationally useful. Resizing, normalization, denoising, enhancement, and thresholding prepare data. Classification, detection, segmentation, pose estimation, optical flow, and tracking infer what is present and how it changes.

Table 7: Visual-processing pipeline for intelligent objects

Stage	Representative operation	Engineering purpose
Acquisition	Exposure, frame rate, sensor interface	Obtain a usable and time-aligned visual measurement.
Preprocessing	Crop, resize, grayscale, normalize	Match model input and reduce compute demand.
Enhancement	Denoising, contrast adjustment	Improve robustness to sensor and lighting conditions.
Segmentation/features	Threshold, edges, contours, keypoints	Isolate regions and construct compact descriptors.
Vision inference	Classification, detection, tracking	Estimate object identity, location, motion, or state.
Decision	Confidence threshold, temporal logic	Convert uncertain predictions into operational state.
Action/connectivity	Motor command, alert, MQTT event	Close the physical loop or coordinate with services.

5.3 Core operations

A common grayscale conversion is

$$I_{\text{gray}} = 0.299R + 0.587G + 0.114B.$$

Binary thresholding produces a region mask:

$$B(x, y) = \begin{cases} 1, & I(x, y) > T, \\ 0, & I(x, y) \leq T. \end{cases}$$

Spatial filtering applies a kernel K by discrete convolution, while edges are related to the image gradient:

$$G(x, y) = \sum_i \sum_j K(i, j) I(x - i, y - j).$$

$$|\nabla I| = \sqrt{I_x^2 + I_y^2}.$$

Deep visual models learn hierarchical features and can be written generally as $\hat{y} = f(I; \theta)$. An object detector returns class, location, and confidence for each candidate. On resource-constrained hardware, input resolution, model topology, operator support, quantization, working memory, and accelerator availability determine whether the model is deployable.

5.4 Edge vision

Sending every frame to a cloud service can be costly and may be unacceptable for latency, connectivity, or privacy. Edge vision performs some or all processing locally and may transmit only events or compact metadata. For example, a parking node can publish `{space:12, occupied:true, confidence:0.94}` instead of continuous video. However, local processing does not automatically guarantee privacy or security; the design must control image retention, access, model leakage, updates, and inference logs.

6. Engineering the Intelligence Lifecycle

6.1 Data-to-action workflow

An IOE workflow begins with the physical mission, not with model selection. The required action, decision deadline, acceptable risk, operating conditions, and failure behavior determine the sensing and intelligence design.

- Define the physical function, stakeholders, operating envelope, hazards, and measurable acceptance criteria.
- Select sensors and actuators; characterize range, resolution, noise, drift, calibration, and timing.
- Collect representative, ethically obtained, and correctly labelled data across expected conditions and failures.
- Design preprocessing and features using the same implementation assumptions intended for deployment.
- Train or configure the inference model; evaluate accuracy, calibration, robustness, bias, and uncertainty.
- Compress and deploy the model; measure memory, latency, energy, and numerical deviation on target hardware.
- Integrate the decision with control, communication, user interaction, and a deterministic safe fallback.
- Monitor field performance, drift, security, and updates throughout the lifecycle.

6.2 Device–edge–cloud partitioning

Table 9: Allocation of intelligence across the computing continuum

Location	Strengths	Limitations and appropriate use
Device	Lowest reaction latency; offline operation; reduced raw-data transfer.	Severe memory, compute, energy, observability, and update constraints.
Gateway/edge	Aggregates nearby devices; stronger models; local coordination.	Additional infrastructure and a possible local single point of failure.
Cloud	Fleet-scale training, storage, analytics, and management.	Network delay, service dependence, bandwidth cost, and data-governance concerns.
Hybrid	Balances fast local response with global learning and management.	Requires explicit ownership of state, synchronization, failure, and versioning.

6.3 Real-time software and communication

Firmware typically separates acquisition, preprocessing, inference, communication, and actuation into scheduled tasks. An RTOS can provide periodic execution, priorities, queues, timers, and mutual exclusion. Design errors may still arise from blocking calls, priority inversion, shared-data races, heap fragmentation, or unbounded communication delays. Message-oriented protocols such as MQTT support telemetry and commands, but topic design, payload schema, quality of service, authentication, stale-message handling, and idempotent actuation must be engineered explicitly.

7. Trustworthiness, Security, and Assurance

An intelligent object can affect the physical world; consequently, model accuracy alone is insufficient evidence of fitness. Assurance must span hardware, firmware, communication, data, models, control, and human interaction. NIST CPS guidance similarly treats trustworthiness, timing, data, composition, and lifecycle as cross-cutting concerns [1], [2].

Table 11: Representative IOE assurance concerns

Concern	Example failure	Engineering response
Safety	Incorrect detection activates a hazardous actuator.	Hazard analysis, interlocks, bounded commands, safe state and independent monitoring.
Security	Spoofed sensor or command changes physical behavior.	Device identity, authenticated encryption, secure boot, signed updates and least privilege.
Privacy	Images or behavioral data expose individuals.	Data minimization, local inference, retention limits, access control and transparency.
Reliability	Sensor drift silently degrades decisions.	Calibration checks, redundancy, plausibility tests, health monitoring and maintenance.

Concern	Example failure	Engineering response
AI robustness	Distribution shift causes overconfident error.	Representative validation, confidence calibration, drift detection and fallback logic.
Timing	Correct result arrives too late for control.	Worst-case timing analysis, deadline monitoring and degradation modes.
Maintainability	Firmware and model versions become incompatible.	Versioned interfaces, reproducible builds, staged OTA rollout and rollback.

8. Illustrative Case Study: Intelligent Plant-Monitoring Object

8.1 Requirements and architecture

Consider an ESP32-class object that monitors soil moisture M , temperature T , relative humidity H , and light level L . It predicts watering demand, controls a low-voltage pump through a protected driver, publishes state by MQTT, and provides manual override. The design objective is not merely to maximize prediction accuracy; it is to keep the plant within an acceptable moisture range while minimizing water and energy use and preventing unsafe pump operation.

$$\mathbf{x}_k = [M_k, T_k, H_k, L_k, \Delta M_k]^T, \quad p_{\text{water}} = f_{\theta}(\mathbf{x}_k).$$

$$u_k = \begin{cases} 1, & p_{\text{water}} > T_p \text{ and safety conditions hold,} \\ 0, & \text{otherwise.} \end{cases}$$

8.2 Operational data flow

- Sensor task: sample and timestamp channels; detect open circuits, saturation, and implausible values.
- Preprocessing task: filter noise, compensate calibration, normalize features, and estimate moisture trend.
- Inference task: calculate watering probability and confidence at a bounded interval.
- Decision task: apply hysteresis, minimum/maximum run time, reservoir state, cooldown, and manual override.
- Communication task: publish measurements, decision reason, confidence, health, and firmware/model versions.
- Actuation task: energize the pump only through a monitored state machine and return to a safe off state on fault.

8.3 Example optimization and validation

A simplified objective can combine water consumption W , moisture tracking error E_m , energy E , and switching count N_{sw} :

$$\min J = \lambda_1 W + \lambda_2 E_m + \lambda_3 E + \lambda_4 N_{\text{sw}}.$$

Validation should include sensor calibration, dry-to-wet transitions, multiple soil types, changing illumination and temperature, network loss, power interruption, empty reservoir, stuck relay, corrupted model, and manual override. System-level metrics include moisture regulation error, water consumed, missed watering events, false activations, reaction time, uptime, energy per day, communication availability, and safe-fallback success—not only classifier accuracy.

9. Application Domains

Table 13: Representative applications of Intelligent Object Engineering

Domain	Intelligent object	Perception–decision–action example
Smart environment	Room controller	Fuse temperature, humidity and occupancy; predict comfort; control ventilation.
Industrial systems	Condition-monitoring node	Analyze vibration; detect anomaly; request maintenance or stop equipment safely.
Wearables	Activity-recognition device	Fuse IMU signals; classify activity; provide feedback or alert.
Agriculture	Plant/irrigation object	Estimate water demand; coordinate pump and publish state.
Mobility Robotics	Parking or traffic node Mobile robot	Detect occupancy/flow; update guidance or signals. Perceive obstacles; localize; plan motion; control motors.
Energy Buildings	Smart meter/controller Visual access object	Forecast demand; detect anomalies; schedule loads. Detect authorized condition; apply policy; actuate lock with audit evidence.

10. Research Challenges and Future Directions

10.1 Co-design under constraints

IOE requires joint hardware–software–model–network–control design. A model that fits in flash may still exceed the RAM arena, miss a real-time deadline, drain the battery, or interact poorly with a controller. Automated design-space exploration that incorporates physical performance and assurance constraints remains an important research direction.

10.2 Continual and collaborative intelligence

Field environments change through sensor ageing, seasonal variation, user behavior, and new operating conditions. Continual learning may mitigate drift but introduces risks of catastrophic forgetting, unverified behavior, and fleet inconsistency. Federated or collaborative learning can reduce centralized raw-data collection, yet it still requires secure aggregation, device heterogeneity management, communication efficiency, and defenses against poisoned updates.

10.3 Explainability and uncertainty at the object level

Resource-constrained explanations must be timely and operationally meaningful. Rather than generating a complex explanation after a decision, an object may report the decisive features, confidence, threshold state, sensor health, and applicable safety rule. Calibrated uncertainty and abstention are particularly important when the physical cost of a false decision is high.

10.4 Digital twins and semantic interoperability

A digital twin can maintain a computational representation of object state, history, and predicted behavior. The challenge is to keep the twin synchronized, semantically compatible, secure, and useful under missing or delayed data. Common data models and explicit state ownership are necessary when many objects and services collaborate.

10.5 Sustainable and responsible design

Energy-aware inference, long-lived hardware, repairability, update support, data minimization, and transparent human control should be treated as design requirements. Intelligence that creates short replacement cycles or unnecessary data collection can undermine the system's claimed benefit.

11. Educational Framework for Engineering

IOE can serve as a curriculum bridge from electronics and MCU programming to cyber-physical intelligence. A progressive course sequence can transform an ordinary embedded object into a connected, intelligent, and assured system.

Table 15: Suggested eight-chapter teaching sequence

Topic	Theme	Laboratory direction
1	Intelligent objects and CPS foundations	ESP32 GPIO, timing, state machines and object capability analysis.
2	Sensors, actuators and data acquisition	ADC, I ² C sensor, calibration, PWM and safe actuation.
3	Real-time embedded computing	FreeRTOS tasks, queues, timers, concurrency and resource measurement.
4	Signals, probability and sensor fusion	Filtering, feature extraction and MPU6050 orientation/activity estimation.
5	Connectivity and connected intelligence	Wi-Fi/BLE, MQTT telemetry, commands and multi-object coordination.
6	Edge AI and TinyML	Dataset creation, quantization, deployment and on-device benchmarking.
7	Image processing and embedded vision	Camera acquisition, preprocessing, classification/detection and event publishing.
8	Security, digital twins and integration	Secure communication/OTA, dashboard or twin, system-level validation.

Assessment should combine theory, implementation, measurement, and reflection. Students should report not only whether a demonstration works, but also timing, memory, energy, uncertainty, network behavior, security assumptions, failure modes, and evidence against requirements.

12. Discussion

The proposed IOE framing clarifies three relationships. First, IoT connectivity is an enabling capability rather than the defining source of intelligence. A connected sensor becomes an intelligent object only when observations are transformed into context-sensitive decisions or coordinated physical functions. Second, embedded AI is one possible intelligence mechanism, not the whole system. Deterministic estimation and control may remain essential around a learned model. Third, image processing is not an isolated application module: it converts high-dimensional sensing into visual state that must still be interpreted, timed, secured, communicated, and connected to safe action.

The framework also exposes the principal trade-off in intelligent-object design. Increasing local autonomy can reduce latency and network dependence, but may reduce observability and restrict model size. Increasing cloud dependence can simplify device computation but introduces latency, availability, privacy, and service-lifecycle concerns. The strongest architecture is therefore rarely ‘all edge’ or ‘all cloud’; it allocates functions according to physical deadlines, resource limits, assurance needs, and fleet-level value.

Because Intelligent Object Engineering is used here as a synthesized academic framework, future work should develop a formal body of knowledge, standard vocabulary, reference implementations, benchmark tasks, and maturity measures. Such work should remain aligned with established CPS, IoT, embedded-systems, AI, control, and systems-engineering standards to avoid unnecessary terminology fragmentation.

13. Conclusion

Intelligent Object Engineering provides an object-centred framework for integrating the physical, computational, intelligent, connected, and lifecycle dimensions of modern engineered systems. An intelligent object senses its environment, processes uncertain data, estimates context, makes a decision, acts within physical constraints, and may coordinate with other objects and services. Its foundation is inherently mathematical, joining dynamic models, linear algebra, probability, signal processing, estimation, optimization, control, information, graphs, and machine learning. Image processing extends this framework by enabling visual perception through cameras and embedded vision. The engineering challenge is to make these capabilities operate together within real-time, resource, safety, security, privacy, and maintainability constraints. This unified view offers a strong foundation for computer-engineering research, curriculum design, and the development of dependable intelligent connected objects.

14. References

- [1] E. R. Griffor, C. Greer, D. A. Wollman, and M. J. Burns, Framework for Cyber-Physical Systems: Volume 1, Overview, NIST Special Publication 1500-201, 2017. doi: 10.6028/NIST.SP.1500-201.
- [2] E. R. Griffor, C. Greer, D. A. Wollman, and M. J. Burns, Framework for Cyber-Physical Systems: Volume 2, Working Group Reports, NIST Special Publication 1500-202, 2017. doi: 10.6028/NIST.SP.1500-202.

- [3] G. Kortuem, F. Kawsar, V. Sundramoorthy, and D. Fitton, “Smart objects as building blocks for the Internet of Things,” *IEEE Internet Computing*, vol. 14, no. 1, pp. 44–51, 2010. doi: 10.1109/MIC.2009.143.
- [4] J. Voas, R. Kuhn, P. Laplante, and S. Applebaum, *Internet of Things (IoT) Trust Concerns*, NIST Cybersecurity White Paper, 2018. doi: 10.6028/NIST.CSWP.16122018.
- [5] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, “Cloud computing and emerging IT platforms,” *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. doi: 10.1109/JIOT.2016.2579198.
- [7] P. P. Ray, “A review on TinyML: State-of-the-art and prospects,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 4, pp. 1595–1623, 2022. doi: 10.1016/j.jksuci.2021.11.019.
- [8] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O’Reilly Media, 2019.
- [9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [10] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [11] S. Haykin, *Kalman Filtering and Neural Networks*. New York, NY, USA: Wiley, 2001.
- [12] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. Princeton, NJ, USA: Princeton University Press, 2021.
- [13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [14] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [15] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2003.
- [16] O. Vermesan and P. Friess, Eds., *Internet of Things: Converging Technologies for Smart Environments and Integrated Ecosystems*. Aalborg, Denmark: River Publishers, 2013.
- [17] J. Lee, B. Bagheri, and H.-A. Kao, “A cyber-physical systems architecture for Industry 4.0-based manufacturing systems,” *Manufacturing Letters*, vol. 3, pp. 18–23, 2015. doi: 10.1016/j.mfglet.2014.12.001.
- [18] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed. Cambridge, MA, USA: MIT Press, 2017.